

Lens K8S IDE

Kubernetes Terms Explained by Improving One Spring Boot Cluster

A beginner-friendly presentation: every term answers
“what is it, when do I use it, how do I configure it,
and where do I see it in Lens?”

Spring Boot API

Kind 1 control-plane + 2
workers

Docker Desktop

kubectl

Lens



How to learn this deck

[Read this before the terms](#)

We will not memorize a menu. We will improve one cluster step by step.

1 Find the app

Overview, Applications, Nodes,
Namespaces, Events

2 Run it correctly

Workloads, Pods, Deployments,
ReplicaSets

3 Choose workload type

DaemonSets, StatefulSets, Jobs,
CronJobs

4 Configure & protect

ConfigMaps, Secrets, Quotas, Limits,
HPA, PDB

5 Expose & debug

Services, EndpointSlices, Ingress,
NetworkPolicy, Port Forward

6 Operate platform

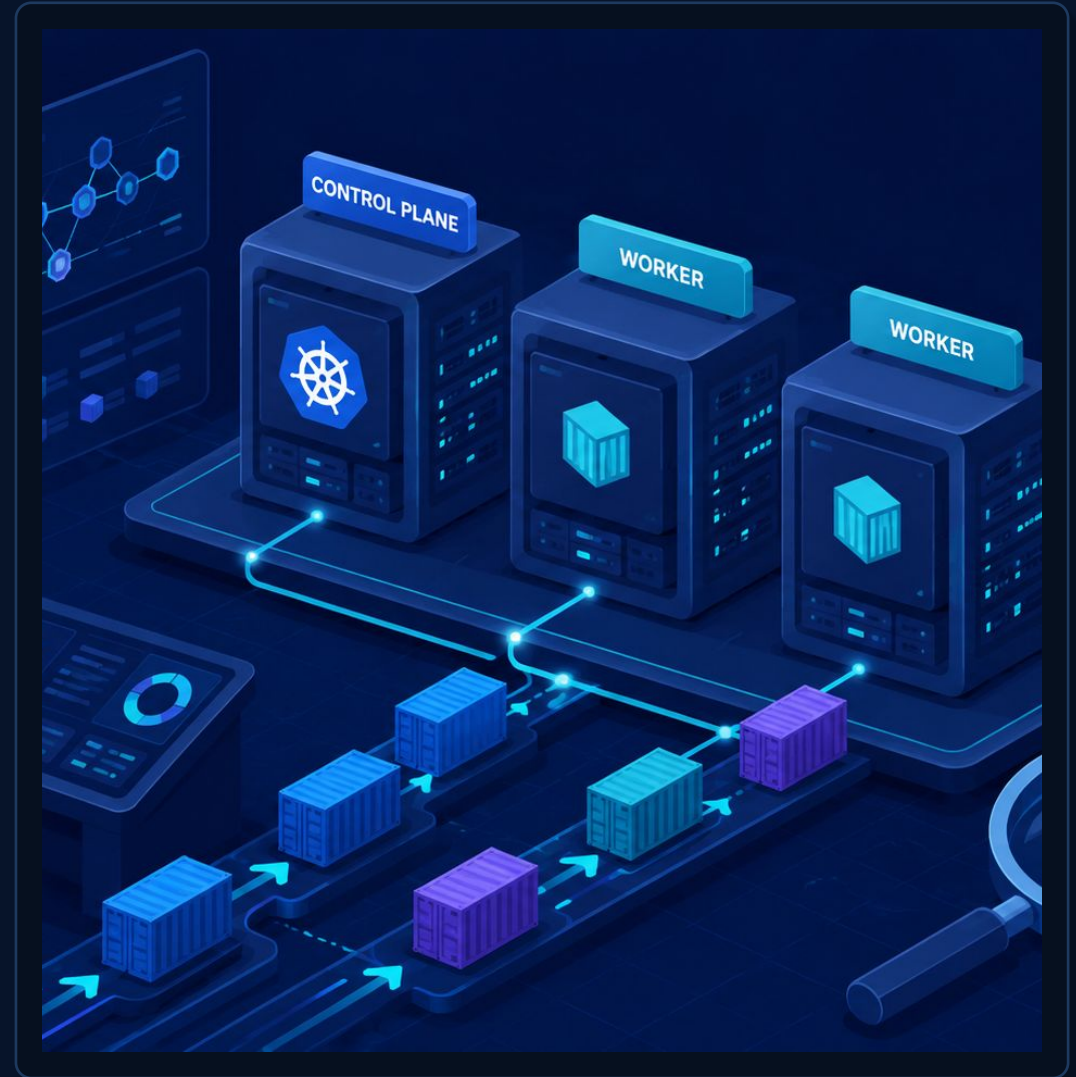
Storage, Helm, RBAC, CRDs, Security
Center

Kind note: “3-node cluster” is 3 Docker containers acting as Kubernetes nodes, not 3 full VMs. That is perfect for learning these Lens views.

Orientation

Stage 1 — Get oriented before touching anything

We have a Spring Boot API in Kind. First, we learn where Lens shows the cluster, the app, the nodes, the namespace and recent events.



Developer situation

“Is my cluster healthy at all?”



Open in Lens

Cluster → Overview

You know whether the problem is cluster-wide or limited to your app.

What is it?

The cluster dashboard that summarizes health, capacity, workloads and resource usage.

When do I use it?

- First screen during every incident.
- Before editing anything in the cluster.
- To notice broad CPU, memory or object problems.

How do I configure / use it?

Open the cluster in Lens and start with Overview. Check node count, status indicators, workload count and metric graphs. If metrics are empty, install Metrics Server.

```
kubectl get nodes  
kubectl get pods -A
```

Developer situation

“Where is my Spring Boot app?”



Open in Lens

Applications

You stop hunting object-by-object and start from the application boundary.

What is it?

A Lens view that groups related Kubernetes objects into an application-level picture.

When do I use it?

- When you want an app-centric view instead of raw Kubernetes objects.
- To see what resources belong together.

How do I configure / use it?

Use Applications to find the demo API, then drill down to the workload, pods, services and configuration behind it. Treat it as the app map.

```
app: spring-demo-api  
namespace: lens-demo
```

Developer situation

“Why do I see one control plane and two workers?”



Open in Lens

Nodes

You understand why the API pods land on worker nodes.

What is it?

A Node is a machine-like worker in Kubernetes. In Kind, each node is a Docker container acting like a Kubernetes node.

When do I use it?

- To check node readiness and capacity.
- To see which pods are on which worker.
- To diagnose scheduling failures.

How do I configure / use it?

Open Nodes. Select control-plane, worker-1 and worker-2. Check labels, taints, conditions, allocatable CPU/memory and pods running on each node.

```
nodeSelector:  
  workload: application
```

Developer situation

“Lens is showing too many resources.”



Open in Lens

Namespaces

You focus only on the resources owned by the demo app.

What is it?

A Namespace is a named boundary that separates resources inside one cluster.

When do I use it?

- To separate demo, staging, production-like resources.
- To filter Lens so you do not confuse app resources with system resources.

How do I configure / use it?

Select the lens-demo namespace in Lens. Inspect namespace labels, quotas, limits and resources that belong to that namespace.

```
apiVersion: v1
kind: Namespace
metadata:
  name: lens-demo
```

Developer situation

**“The pod is not ready, but I
do not
know why.”**



Open in Lens

Events

You hear Kubernetes explain the
failure before guessing.

What is it?

**Events are short records of
important cluster
changes, warnings and
scheduling decisions.**

When do I use it?

- When a Pod is Pending, CrashLooping, or failing probes.
- After scaling, rollout, deletion or image changes.

How do I configure / use it?

Open Events, filter to lens-demo, sort by newest. Read messages like Scheduled, Pulling, Started, Unhealthy or FailedScheduling.

```
kubectl get events -n lens-demo  
--sort-by=.lastTimestamp
```


Workloads

Stage 2 — Run the Spring Boot API correctly

Now the developer learns the workload objects: the running Pod, the Deployment that owns it, and the replica objects in between.



Developer situation

“What is running in this namespace?”



Open in Lens

Workloads → Overview

You know whether the problem is a web app, background job, or platform component.

What is it?

A summary page for workload types: Pods, Deployments, DaemonSets, StatefulSets, Jobs and CronJobs.

When do I use it?

- To quickly see workload health.
- Before deciding which controller object to inspect.

How do I configure / use it?

Open Workloads Overview. Start with unhealthy counts, then drill into the exact workload kind that owns the problem.

```
kubectl get workloads -n lens-demo  
# Lens provides the visual grouping
```

Developer situation

“What is actually running my API code?”



Open in Lens

Workloads → Pods

You understand the runtime result, but you do not edit Pods as the main fix.

What is it?

A Pod is the smallest deployable Kubernetes unit. It runs one or more containers together with shared network and volumes.

When do I use it?

- To view logs and container status.
- To inspect node placement, restarts, probes and environment.
- To open a shell for debugging.

How do I configure / use it?

Open a Pod. Check status, restarts, logs, events, container image, env vars, probes, volumes and node name. Delete a pod only to test self-healing.

```
apiVersion: v1
kind: Pod
# Usually created by Deployment,
# not manually for web APIs.
```

Deployments

2.3 Desired state for
stateless APIs

Developer situation

**“I need two API replicas
and safe
rolling updates.”**



Open in Lens

Workloads → Deployments

You control the app at the correct
owner level.

What is it?

**A Deployment manages
stateless application
replicas and updates them
through ReplicaSets.**

When do I use it?

- For Spring Boot APIs and web apps.
- When pods can be replaced without losing local state.
- When you need rolling updates and rollback.

How do I configure / use it?

Open the Deployment. Inspect replicas, image, rollout strategy, env, probes, resources, node selector and events. Edit the Deployment, not individual Pods.

```
kind: Deployment
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
```

Developer situation

“Who recreated the Pod after I deleted it?”



Open in Lens

Workloads → Replica Sets

You learn why deleting a Pod does not reduce the desired replica count.

What is it?

A ReplicaSet keeps a specified number of matching Pods running.

When do I use it?

- Mostly to understand what a Deployment created.
- To inspect rollout history and current vs old replica groups.

How do I configure / use it?

Open ReplicaSets under the Deployment. The current ReplicaSet has active replicas; older ones may stay with zero replicas for rollback history.

Deployment → ReplicaSet → Pod
Usually do not create directly

ReplicaControllers

2.5 Legacy replica controller

Developer situation

“Why is this old object in Lens?”



Open in Lens

Workloads → Replica Controllers

You recognize it without choosing it for the new Spring Boot API.

What is it?

A ReplicationController is the older object that kept a number of Pods running before ReplicaSets and Deployments became standard.

When do I use it?

- Only when maintaining legacy clusters.
- To understand old manifests or migrated apps.

How do I configure / use it?

If Lens shows one, inspect it like a legacy owner object. For new work, prefer Deployment for stateless apps.

```
kind: ReplicationController
# Legacy; prefer Deployment
```

Pod vs Deployment vs ReplicaSet vs ReplicaController

2.6 Workload ownership

Use this comparison when you are unsure which object you should edit in Lens.

Pod

Runs the container now.

Observe/debug only

Deployment

Defines desired app state.

Edit for API changes

ReplicaSet

Keeps replica count.

Understand rollout

ReplicaController

Legacy replica keeper.

Avoid for new apps

Workload patterns

Stage 3 — Pick the right workload pattern

Now we meet workloads that are not a normal stateless API: node agents, stateful apps and batch work.



Developer situation

“I need an agent on every node.”



Open in Lens

Workloads → Daemon Sets

You know why system agents appear on each Kind node.

What is it?

A DaemonSet runs a copy of a Pod on selected nodes, commonly one per node.

When do I use it?

- For log collectors, monitoring agents, storage agents, network plugins.
- When every node needs the same helper process.

How do I configure / use it?

Open DaemonSets to see desired, current and ready pods. Check whether node selectors or tolerations allow it on the intended nodes.

```
kind: DaemonSet
spec:
  selector:
    matchLabels:
      app: node-agent
```

Developer situation

“A database pod needs a stable name and disk.”



Open in Lens

Workloads → Stateful Sets

You do not use Deployment when the app relies on stable identity.

What is it?

A StatefulSet manages Pods that need stable network identity, ordered rollout and persistent storage identity.

When do I use it?

- For databases, brokers and clustered systems.
- When pod identity matters: app-0, app-1, app-2.

How do I configure / use it?

Open StatefulSets. Check replicas, ordinal pods, PVC templates, update strategy and each pod's attached storage.

```
kind: StatefulSet
serviceName: demo-db
volumeClaimTemplates: [...]
```

Developer situation

“I need to run a database migration once.”



Open in Lens

Workloads → Jobs

You separate short-lived work from long-running API Pods.

What is it?

A Job creates Pods and expects them to finish successfully.

When do I use it?

- For one-time migrations, imports, reports and batch commands.
- When success/failure matters more than always-running service availability.

How do I configure / use it?

Open Jobs. Check completions, failed pods, logs, duration and events. Delete old demo jobs when they are no longer useful.

```
kind: Job
spec:
  template:
    spec:
      restartPolicy: Never
```

Developer situation

“I need a health check or cleanup every 5 minutes.”



Open in Lens

Workloads → Cron Jobs

You understand scheduled automation inside Kubernetes.

What is it?

A CronJob creates Jobs on a schedule using cron syntax.

When do I use it?

- For scheduled cleanup, reconciliation, reports, backups or health probes.
- When work should run repeatedly but not constantly.

How do I configure / use it?

Open CronJobs. Check schedule, suspend flag, last schedule time and the Jobs it created. Then inspect a Job Pod log.

```
kind: CronJob
schedule: "*/5 * * * *"
```

Which workload should the developer use?

Use this comparison when you are unsure which object you should edit in Lens.

Deployment

Stateless API or web app.

spring-demo-api

DaemonSet

One agent on each node.

logs / monitoring

StatefulSet

Stable name + storage.

database / broker

Job

Run once and finish.

migration

CronJob

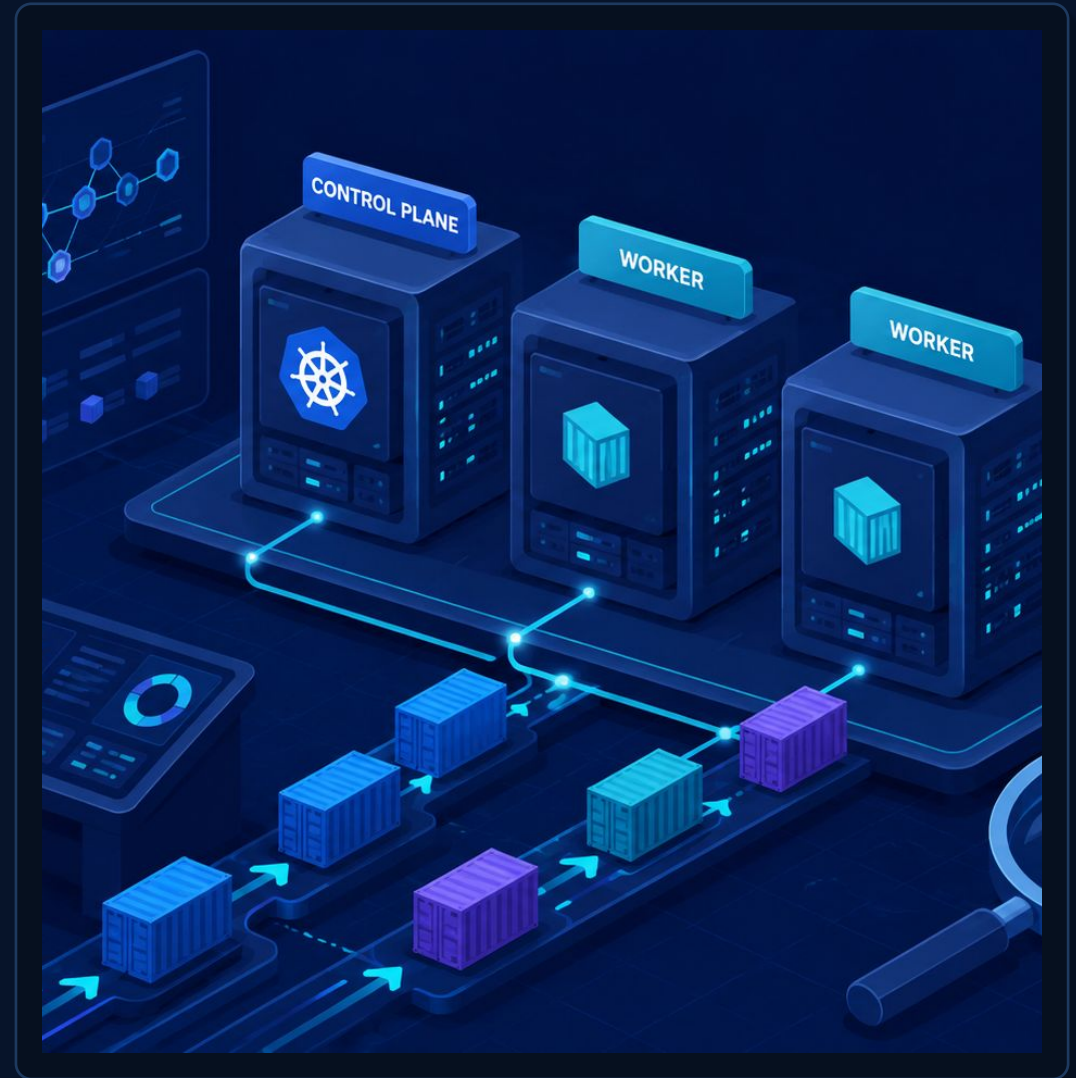
Run on a schedule.

cleanup / report

Configuration

Stage 4 — Configure the app without rebuilding it

The developer separates configuration from container image, then adds safety limits and resilience controls.



Developer situation

“I want to change the greeting without rebuilding the image.”



Open in Lens

Config → Config Maps

Configuration becomes environment-specific instead of baked into the image.

What is it?

A ConfigMap stores non-confidential key-value configuration or small config files.

When do I use it?

- For environment names, feature flags, URLs and non-secret settings.
- When the same image needs different behavior per environment.

How do I configure / use it?

Open ConfigMaps. Edit non-secret values. If mounted as env vars, restart the Deployment; if mounted as volume, the app may need to reload the file.

```
kind: ConfigMap
data:
  DEMO_GREETING: "Hello"
```

Developer situation

“The API needs a token but must not expose it.”



Open in Lens

Config → Secrets

You know how secret references work and why production secret handling requires discipline.

What is it?

A Secret stores sensitive data such as tokens, passwords or certificates as Kubernetes secret objects.

When do I use it?

- For credentials and private values.
- When a value must not be displayed by the API or committed as plain text.

How do I configure / use it?

Open Secrets to inspect key names and references. For production, prefer external secret management or encryption workflows instead of plain YAML.

```
kind: Secret
type: Opaque
stringData:
  DEMO_SECRET: change-me
```


Developer situation

**“Scaled too many pods
and
consumed the cluster.”**



Open in Lens

Config → Resource Quotas

The namespace has a maximum
blast radius.

What is it?

**A ResourceQuota limits total
resource usage
inside a namespace.**

When do I use it?

- To cap pods, CPU, memory, services or storage.
- To prevent one namespace from starving others.

How do I configure / use it?

Open Resource Quotas. Check used vs hard limits. If pods stay Pending, compare their requests with remaining quota.

```
kind: ResourceQuota
spec:
  hard:
    pods: "20"
    requests.cpu: "2"
```

Limit Ranges

Developer situation

“Developers forgot CPU and memory requests.”



Open in Lens

Config → Limit Ranges

Every container gets baseline resource behavior.

What is it?

A LimitRange sets default and allowed resource requests/limits for objects in a namespace.

When do I use it?

- To make missing requests less dangerous.
- To prevent tiny or huge resource declarations.

How do I configure / use it?

Open LimitRanges. Check defaultRequest, default and max/min values. Remember explicit values in a Pod spec override defaults.

```
kind: LimitRange
spec:
  limits:
    - type: Container
      defaultRequest:
        cpu: 100m
```

Horizontal Pod Autoscalers

4.5 Scale based on metrics

Developer situation

“Traffic spikes and I do not want manual scaling.”



Open in Lens

Config → Horizontal Pod Autoscalers

The API scales within controlled boundaries.

What is it?

An HPA automatically changes replica count based on metrics such as CPU utilization.

When do I use it?

- For stateless workloads that can safely run multiple replicas.
- When you have Metrics Server or another metrics source.

How do I configure / use it?

Open HPA. Watch current metric, target, min replicas, max replicas and scaling events. Avoid fighting it with manual replica edits.

```
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 4
```

Pod Disruption Budgets

4.6 Safer voluntary disruption

Developer situation

“Maintenance should not evict all API pods at once.”



Open in Lens

Config → Pod Disruption Budgets

Planned operations respect minimum availability.

What is it?

A PDB tells Kubernetes how many Pods must remain available during voluntary disruptions.

When do I use it?

- For replicated apps during node drain or maintenance.
- When availability matters during planned operations.

How do I configure / use it?

Open PDB. Check desired healthy, current healthy and allowed disruptions. It does not protect against all crashes.

```
kind: PodDisruptionBudget
spec:
  minAvailable: 1
```

Developer situation

“Some pods are more important when capacity is tight.”



Open in Lens

Config → Priority Classes

You understand why some Pods win scheduling decisions.

What is it?

A PriorityClass defines scheduling priority. Higher priority Pods can be scheduled before lower priority Pods.

When do I use it?

- For critical platform workloads.
- When cluster capacity pressure requires ordering.
- Use carefully; wrong priority can hurt normal apps.

How do I configure / use it?

Open Priority Classes to see available names and values. Assign `priorityClassName` only when the platform team defines the policy.

```
priorityClassName: high-priority
```

Developer situation

“A workload needs a special runtime sandbox.”



Open in Lens

Config → Runtime Classes

You recognize runtime selection as an advanced cluster capability.

What is it?

A RuntimeClass selects a configured container runtime handler for a Pod.

When do I use it?

- For sandboxed runtimes, GPU/runtime variants or special isolation.
- Only when the cluster has runtime handlers configured.

How do I configure / use it?

Open Runtime Classes. If none exist in Kind, that is normal. In real clusters, check handler names before using runtimeClassName.

```
runtimeClassName: gvisor  
# only if configured
```

Developer situation

**“How does Kubernetes
know who is
leader or alive?”**



Open in Lens

Config → Leases

You see coordination objects without
confusing them with app config.

What is it?

**A Lease is a lightweight
coordination object
often used for leader
election and node
heartbeats.**

When do I use it?

- To inspect control-plane or controller coordination.
- When troubleshooting leader election in operators/controllers.

How do I configure / use it?

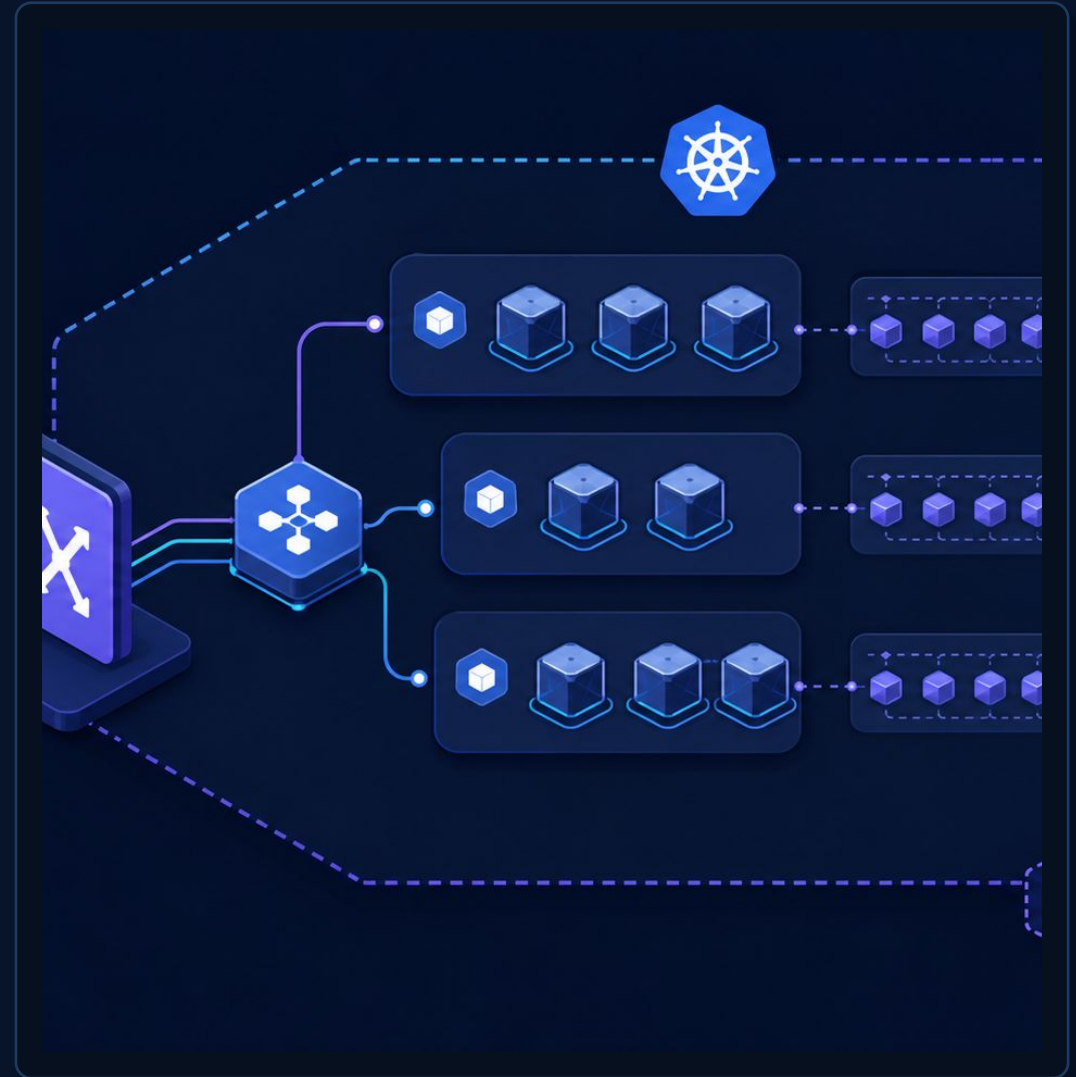
Open Leases, usually in kube-system or controller namespaces. Do not edit them casually; observe renew time and holder identity.

```
kind: Lease
spec:
  holderIdentity: controller-1
```

Network

Stage 5 — Make traffic understandable

The developer follows the path from browser to API Pod and learns how Lens exposes networking objects.



Developer situation

“Pods change IPs, but users need one stable address.”



Open in Lens

Network → Services

You understand how localhost reaches the Spring Boot API.

What is it?

A Service gives a stable virtual address and load-balances traffic to matching Pods.

When do I use it?

- For internal service discovery.
- For exposing Pods through ClusterIP, NodePort or LoadBalancer.
- For stable access to changing replicas.

How do I configure / use it?

Open Services. Check type, ClusterIP, port, targetPort, selector and backing endpoints. For Kind demo, NodePort maps to localhost through Kind port mapping.

```
kind: Service
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
```

Developer situation

**“Service exists, but which
Pods are
behind it?”**



Open in Lens

Network → Endpoint Slices

You diagnose “Service has no
healthy backend.”

What is it?

**EndpointSlices list the
network endpoints
behind a Service in a
scalable way.**

When do I use it?

- To debug whether Service selectors match ready Pods.
- To see backend IPs, ports and readiness.

How do I configure / use it?

Open EndpointSlices for spring-demo-api.
Confirm there are two ready endpoint
addresses when two API Pods are ready.

```
kind: EndpointSlice
# Created by Kubernetes
# from matching Service + Pods
```

Developer situation

**“Why are there both
Endpoints and
EndpointSlices?”**



Open in Lens

Network → Endpoints

You recognize both views as service
backend discovery.

What is it?

**Endpoints is the older object
that stores
backend addresses for a
Service.**

When do I use it?

- For legacy tools or simple debugging.
- When a cluster or integration still reads Endpoints.

How do I configure / use it?

Open Endpoints and compare with
EndpointSlices. Newer clusters prefer
EndpointSlices, but Lens may show both.

```
kind: Endpoints  
# Legacy / compatibility view
```

Developer situation

**“NodePort is okay locally,
but I need
host/path routing.”**



Open in Lens

Network → Ingresses

You separate HTTP routing from the
Service object.

What is it?

**An Ingress defines
HTTP/HTTPS routing rules
from outside the cluster to
Services.**

When do I use it?

- For host-based or path-based web routing.
- When you have an ingress controller installed.
- For TLS termination patterns.

How do I configure / use it?

Open Ingresses. Check host, paths, target Services, TLS and events. Remember: Ingress needs a controller to actually work.

```
kind: Ingress
spec:
  rules:
    - host: demo.local
```

Developer situation

**“There are multiple
ingress
controllers.”**



Open in Lens

Network → Ingress Classes

Your Ingress rule is handled by the
intended controller.

What is it?

**An IngressClass identifies
which controller
should handle a given
Ingress.**

When do I use it?

- When clusters have nginx, traefik, cloud LB or other controllers.
- To make routing ownership explicit.

How do I configure / use it?

Open Ingress Classes. Check controller field.
In your Ingress, use ingressClassName
matching the installed controller.

```
ingressClassName: nginx
```

Developer situation

**“Only approved Pods
should reach the
API.”**



Open in Lens

Network → Network Policies

Network access becomes
intentional, not accidental.

What is it?

**A NetworkPolicy controls
allowed ingress and
egress traffic for selected
Pods.**

When do I use it?

- To restrict Pod-to-Pod communication.
- For zero-trust-style namespace security.
- Only effective when the cluster CNI supports it.

How do I configure / use it?

Open Network Policies. Check podSelector, ingress rules and egress rules. Test traffic before and after applying policies.

```
kind: NetworkPolicy
spec:
  podSelector:
    matchLabels:
      app: spring-demo-api
```

Port Forwarding

5.7 Temporary local tunnel

Developer situation

**“I need local debug access
without
changing Service
exposure.”**



Open in Lens

Network → Port Forwarding

You debug safely without changing
cluster networking.

What is it?

**Port forwarding creates a
temporary tunnel
from your machine to a Pod
or Service.**

When do I use it?

- For debugging private Services.
- When you do not want to expose a permanent external port.

How do I configure / use it?

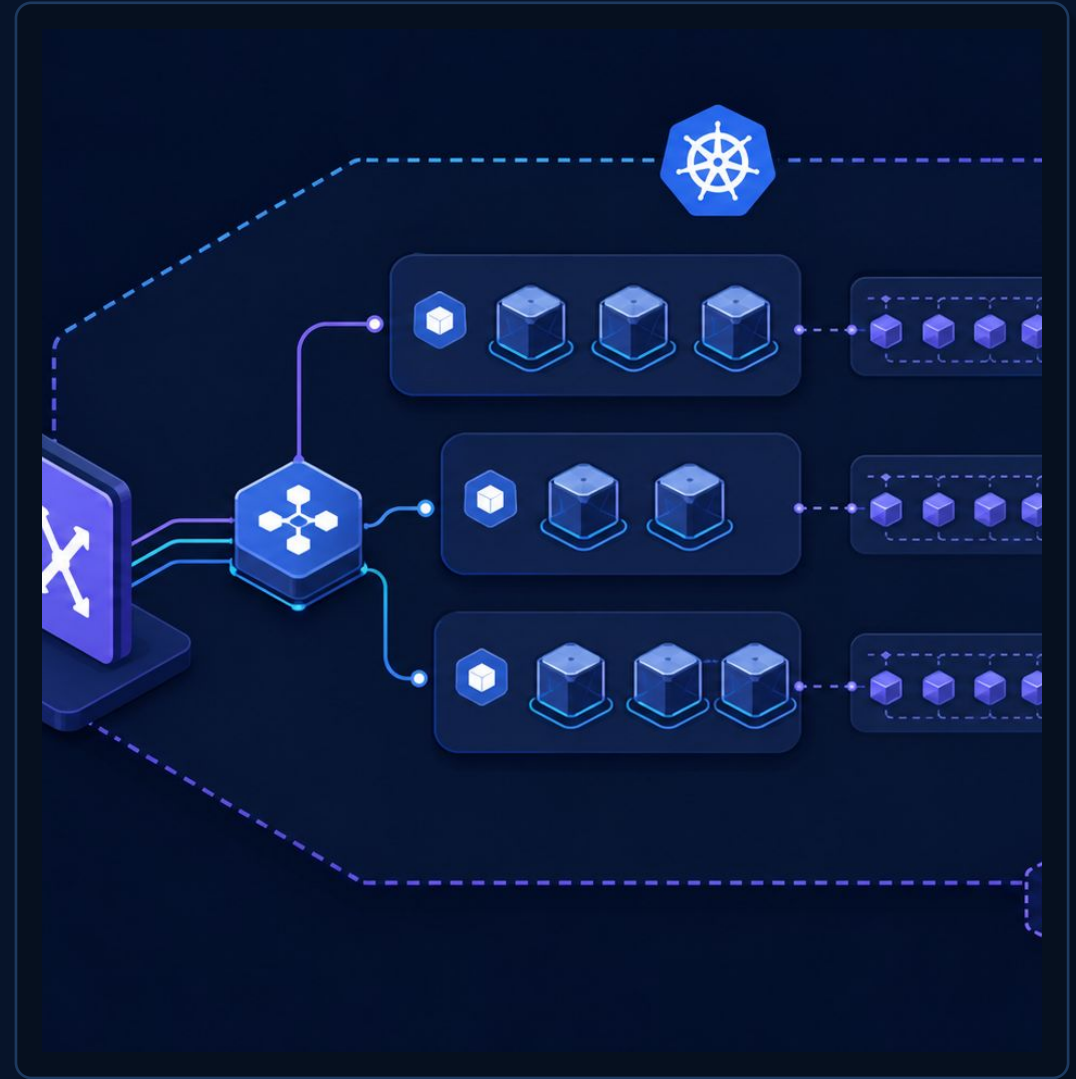
In Lens, start port-forward from a Pod or Service, choose local port, test in browser, then stop the session.

```
kubectl port-forward -n lens-demo  
service/spring-demo-api 18080:80
```

Storage

Stage 6 — Add storage consciously

The developer learns that Pods are replaceable, while persistent data needs a separate storage contract.



Persistent Volume Claims

Developer situation

“A future database needs disk that survives Pod recreation.”



Open in Lens

Storage → Persistent Volume Claims

The app asks for storage without knowing the physical disk details.

What is it?

A PVC is an application request for persistent storage with size and access mode.

When do I use it?

- For databases, file uploads, queues or stateful workloads.
- When data must survive Pod deletion.

How do I configure / use it?

Open PVCs. Check status Bound/Pending, requested size, access mode, StorageClass, bound PV and mounted Pods.

```
kind: PersistentVolumeClaim
spec:
  resources:
    requests:
      storage: 1Gi
```

Developer situation

“What storage satisfied the claim?”



Open in Lens

Storage → Persistent Volumes

You see the real storage object behind the application request.

What is it?

A PV represents actual cluster storage that can be bound to a PVC.

When do I use it?

- To debug Pending PVCs.
- To inspect reclaim policy, capacity, access modes and backing source.

How do I configure / use it?

Open PVs. Check which PVC is bound, reclaim policy and capacity. In dynamic provisioning, PVs may be created automatically.

```
kind: PersistentVolume
spec:
  capacity:
    storage: 1Gi
```

Developer situation

**“Who creates the disk
when a PVC asks
for one?”**



Open in Lens

Storage → Storage Classes

You understand the storage policy
behind PVC creation.

What is it?

**A StorageClass defines a
storage provisioning
type and parameters.**

When do I use it?

- For dynamic storage provisioning.
- To choose fast, cheap, replicated or local storage classes.

How do I configure / use it?

Open Storage Classes. Check provisioner, default class marker, volumeBindingMode and reclaim policy. Use storageClassName in PVCs.

```
storageClassName: standard
```

Helm

Stage 7 — Install and manage packaged components

The developer stops copy-pasting every manifest and learns how packaged Kubernetes software appears in Lens.



Developer situation

“I want to install a standard tool repeatably.”



Open in Lens

Helm → Charts

You treat complex YAML as a managed package.

What is it?

A Helm Chart is a package of Kubernetes templates plus default values.

When do I use it?

- To install common tools like ingress, metrics, monitoring or databases.
- When you need configurable, repeatable installs.

How do I configure / use it?

Open Charts in Lens to browse or install charts from configured repositories. Review values before install.

```
helm install my-release chart-name  
--values values.yaml
```

Developer situation

**“What did that chart install
and how
do I upgrade it?”**



Open in Lens

Helm → Releases

You operate the installed package
as one unit.

What is it?

**A Helm Release is one
installed instance of a
chart with chosen values
and version history.**

When do I use it?

- To inspect installed packages.
- To upgrade, rollback or uninstall a packaged component.

How do I configure / use it?

Open Releases. Check chart version, app version, values, resources, notes and revision history.

```
helm upgrade my-release chart-name  
helm rollback my-release 1
```

Access Control

Stage 8 — Give the app minimum permissions

The developer learns Kubernetes identity and RBAC: who is allowed to do what, and where.



Developer situation

“What identity does my Pod use?”



Open in Lens

Access Control → Service Accounts

Your app identity is explicit and minimal.

What is it?

A ServiceAccount is an identity for processes running inside Pods.

When do I use it?

- When a workload must call the Kubernetes API.
- To avoid using the default identity for every app.

How do I configure / use it?

Open Service Accounts. Check which Pods use each one. In Deployment, set `serviceAccountName` and disable token mounting when not needed.

```
serviceAccountName: spring-demo-api
automountServiceAccountToken: false
```


Developer situation

“The app needs permission only inside lens-demo.”



Open in Lens

Access Control → Roles

Permissions stay inside the namespace boundary.

What is it?

A Role grants permissions to resources inside one namespace.

When do I use it?

- For namespace-scoped access.
- When the permission should not apply cluster-wide.

How do I configure / use it?

Open Roles. Read apiGroups, resources and verbs. Use least privilege: only the resources and verbs the workload needs.

```
kind: Role
rules:
- resources: ["pods"]
  verbs: ["get", "list"]
```

Developer situation

“A controller needs to read nodes across the cluster.”



Open in Lens

Access Control → Cluster Roles

You know when permissions escape one namespace.

What is it?

A ClusterRole grants cluster-wide permissions or reusable permission sets.

When do I use it?

- For cluster-scoped resources like Nodes or PVs.
- For platform components and controllers.
- Use carefully; blast radius is larger.

How do I configure / use it?

Open Cluster Roles. Inspect verbs and resources. Avoid wildcard permissions unless truly required.

```
kind: ClusterRole
rules:
- resources: ["nodes"]
  verbs: ["get", "list"]
```

Role Bindings

8.4 Attach a Role to a subject

Developer situation

“The Role exists, but nobody receives it.”



Open in Lens

Access Control → Role Bindings

Permission chain becomes complete: subject → binding → role.

What is it?

A RoleBinding attaches a Role or ClusterRole to users, groups or ServiceAccounts inside a namespace.

When do I use it?

- To grant namespace-scoped access.
- When binding app ServiceAccount to a Role.

How do I configure / use it?

Open RoleBindings. Check subjects and roleRef. The binding is where you answer “who gets this permission?”

```
kind: RoleBinding
subjects:
- kind: ServiceAccount
  roleRef:
    kind: Role
```

Cluster Role Bindings

Developer situation

“A platform component needs cluster-wide access.”



Open in Lens

Access Control → Cluster Role Bindings

You can identify risky broad permissions.

What is it?

A ClusterRoleBinding attaches a ClusterRole to subjects at cluster scope.

When do I use it?

- For cluster administrators and platform controllers.
- When the subject truly needs access across namespaces.

How do I configure / use it?

Open ClusterRoleBindings. Audit broad bindings carefully, especially cluster-admin or wildcard rules.

```
kind: ClusterRoleBinding
roleRef:
  kind: ClusterRole
```

RBAC chain in plain English

Use this comparison when you are unsure which object you should edit in Lens.

ServiceAccount

Who the Pod is.

identity

Role

What can be done in one namespace.

permission

ClusterRole

What can be done cluster-wide.

permission

RoleBinding

Who gets a Role in namespace.

assignment

ClusterRoleBinding

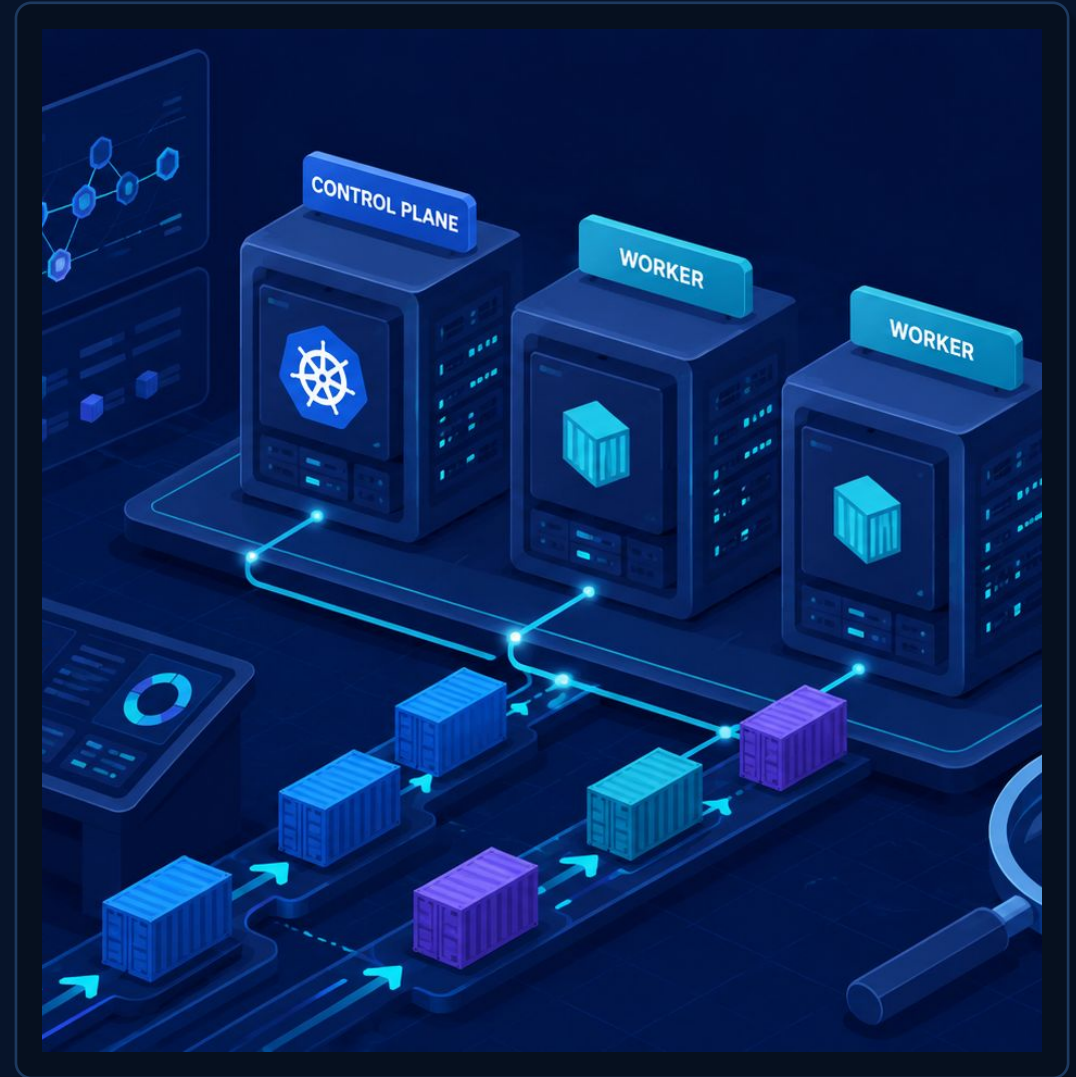
Who gets cluster-wide Role.

assignment

Custom Resources

Stage 9 — Recognize platform extensions

The developer installs tools that add new Kubernetes object types beyond the built-in list.



Developer situation

“A tool installed new object types I have never seen.”



Open in Lens

Custom Resources → Definitions

You understand how Kubernetes grows beyond built-in objects.

What is it?

A CRD defines a new Kubernetes resource type added by an operator or platform extension.

When do I use it?

- When installing cert-manager, monitoring operators, service meshes or custom platforms.
- To inspect new API kinds and versions.

How do I configure / use it?

Open Custom Resources → Definitions.
Check group, version, scope and names.
Then inspect actual custom resources created from that definition.

```
kind: CustomResourceDefinition
spec:
  group: example.com
  names:
    kind: DemoApp
```

Security Center

Stage 10 — Review security before calling it done

The developer checks whether the running cluster is also reasonably safe. Some Security Center features depend on Lens plan and permissions.



Developer situation

“The app works, but is the cluster risky?”



Open in Lens

Security Center → Overview

You stop treating “running” as the same thing as “safe.”

What is it?

A Lens security summary view for detected risks across images, resources and roles when available.

When do I use it?

- Before demo completion or release review.
- When you want a single place to start security triage.

How do I configure / use it?

Open Security Center Overview. Review the highest severity findings first, then drill into Images, Resources or Roles.

Security review order:

1. Critical images
2. Privileged resources
3. Broad roles

Developer situation

“Could my image contain known vulnerabilities?”



Open in Lens

Security Center → Images

Image hygiene becomes part of the developer workflow.

What is it?

The Images view reports detected vulnerability findings for container images when Lens security reporting is enabled.

When do I use it?

- After building or upgrading images.
- Before using base images in demos or production.

How do I configure / use it?

Open Images, sort by severity, review affected image tags, then rebuild or patch base images as needed.

Fix pattern:
FROM eclipse-temurin:21-jre
rebuild when base has patches

Developer situation

“Are my workload settings unsafe?”



Open in Lens

Security Center → Resources

Security fixes are applied to the owner object, not patched randomly on running Pods.

What is it?

The Resources view highlights risky Kubernetes object configurations when the feature is available.

When do I use it?

- To find privileged containers, weak security context, missing limits or exposed resources.
- During platform review.

How do I configure / use it?

Open Resources. Drill into findings, then fix the owning controller such as Deployment or DaemonSet.

```
securityContext:  
  runAsNonRoot: true  
  allowPrivilegeEscalation: false
```

Developer situation

“Did I accidentally grant too much access?”



Open in Lens

Security Center → Roles

RBAC risk becomes visible instead of hidden in YAML.

What is it?

The Roles security view helps identify risky RBAC permissions and broad role bindings when available.

When do I use it?

- Before giving apps Kubernetes API access.
- When auditing cluster-admin, wildcard verbs or broad subjects.

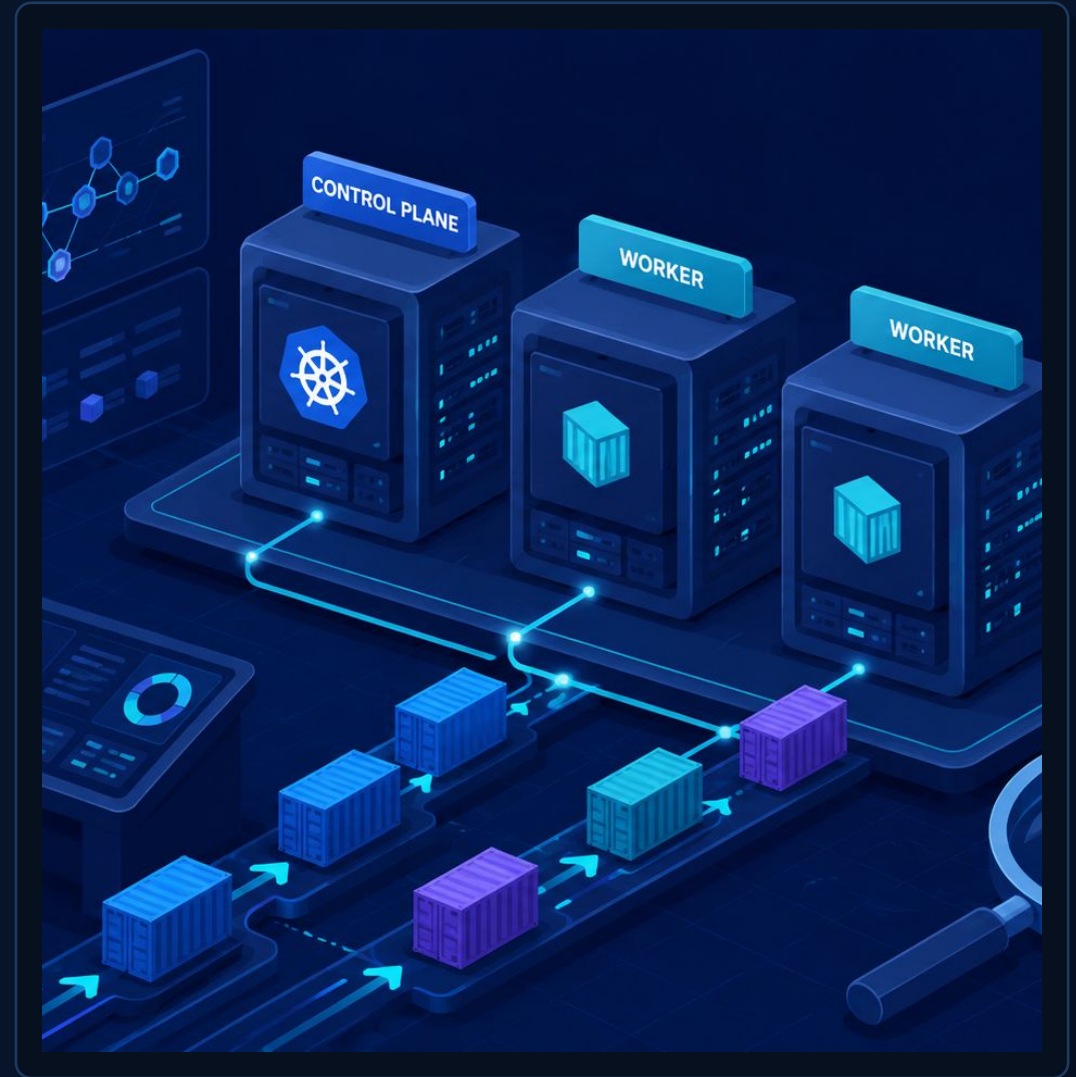
How do I configure / use it?

Open Roles findings, then inspect the related Role, ClusterRole and binding. Replace wildcards with exact resources and verbs.

Avoid:
verbs: ["*"]
resources: ["*"]

Final — developer's Lens routine

The same routine works whether you are debugging Spring Boot, database, ingress, Helm, RBAC or storage issues.



Start wide, then narrow down

Final runbook

Use this comparison when you are unsure which object you should edit in Lens.

Overview

Is the cluster broadly healthy?

first look

Namespace

Am I in the right scope?

filter

Workloads

What controller owns the problem?

owner

Pods + Events

What actually failed?

evidence

Network / Config / Storage / RBAC

Which layer owns the fix?

root cause

Security Center

Is it safe enough?

review

Reference sources used to validate terminology

Sources

Lens docs

docs.k8slens.dev — Lens K8S IDE documentation and how-to pages

Lens feature list

lenshq.io/pricing — Applications, Nodes, Workloads, Configuration, Network, Storage, Helm, Port forwarding, Security Center

Kubernetes docs

kubernetes.io/docs — Workloads, Services, Storage, RBAC, Config, Policy, Extend Kubernetes

RBAC docs

kubernetes.io/docs/reference/access-authn-authz/rbac/ — Role, ClusterRole, RoleBinding, ClusterRoleBinding

Practical warning: some Lens capabilities, especially Security Center reporting, can depend on Lens subscription/plan and cluster permissions.